

scenario1_analysis

May 19, 2026

1 Scenario 1: Operational Baseline Analysis

The extracted Zeek metadata is then ingested into a Python data analysis environment utilizing the Pandas library. In this phase, the raw datasets are cleansed of environmental background noise, such as standard Mininet overhead and Layer 2 broadcast traffic. The dataset is filtered to isolate the flows specifically associated with the target IoT IP addresses and the gateway. Finally, critical behavioral metrics are mathematically derived from the timestamps, most notably the Inter-Arrival Time (IAT) between specific device communications.

Topology Definition: * Smart Camera: 10.0.0.1 * Smart Thermostat: 10.0.0.2 * Attacker Node: 10.0.0.3 * Cloud/Gateway: 10.0.0.254 * Duration: 1200 seconds (20 minutes)

```
[1]: import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import seaborn as sns
from zat.log_to_dataframe import LogToDataFrame

# Set academic plotting style for LaTeX embedding
sns.set_theme(style="whitegrid")
plt.rcParams.update({
    "font.family": "serif",
    "axes.labelsize": 12,
    "font.size": 12,
    "legend.fontsize": 10,
    "xtick.labelsize": 10,
    "ytick.labelsize": 10,
    "figure.dpi": 300
})

# Define paths
LOG_DIR = "../../zeek_logs/scenario1_logs/" # Adjust relative path if needed
OUTPUT_DIR = "../../figures/scenario1/"
os.makedirs(OUTPUT_DIR, exist_ok=True)
```

```

def save_figure(fig, filename_base, output_dir=OUTPUT_DIR):
    pdf_path = os.path.join(output_dir, f"{filename_base}.pdf")
    png_path = os.path.join(output_dir, f"{filename_base}.png")
    fig.savefig(pdf_path, format="pdf", bbox_inches="tight")
    fig.savefig(png_path, format="png", bbox_inches="tight", dpi=300)
    print(f"Saved figure: {pdf_path} and {png_path}")

def format_elapsed_mmss(value, _):
    total_seconds = max(0, int(round(value * 60)))
    minutes, seconds = divmod(total_seconds, 60)
    return f"{minutes:02d}:{seconds:02d}"

```

1.1 Data Ingestion & Preprocessing

We utilize the Zeek Analysis Tools (zat) library to safely parse the TSV logs, automatically handling the commented headers and converting Zeek's epoch timestamps into native Python Datetime indices. We then filter out any non-IoT subnet noise (e.g., Mininet IPv6/multicast background traffic).

```

[2]: # Initialize ZAT parser
log_to_df = LogToDataFrame()

# Ingest Zeek Logs
conn_df = log_to_df.create_dataframe(os.path.join(LOG_DIR, 'conn.log'))
dns_df = log_to_df.create_dataframe(os.path.join(LOG_DIR, 'dns.log'))
ntp_df = log_to_df.create_dataframe(os.path.join(LOG_DIR, 'ntp.log'))

# Deduplicate packets caused by Mininet capturing on 'any' interface
conn_df = conn_df.drop_duplicates(subset=['uid'], keep='first')
dns_df = dns_df.drop_duplicates(subset=['uid'], keep='first')
ntp_df = ntp_df.drop_duplicates(subset=['uid'], keep='first')

# Preprocessing: Ensure timestamps are timezone-naive for easier math
if conn_df.index.tz is not None:
    conn_df.index = conn_df.index.tz_localize(None)
if dns_df.index.tz is not None:
    dns_df.index = dns_df.index.tz_localize(None)
if ntp_df.index.tz is not None:
    ntp_df.index = ntp_df.index.tz_localize(None)

# Define our forensic scope
target_ips = ['10.0.0.1', '10.0.0.2', '10.0.0.3', '10.0.0.254']

# Filter logs to ensure we capture any conversation touching our target
↳ architecture

```

```
conn_df = conn_df[conn_df['id.orig_h'].isin(target_ips) | conn_df['id.resp_h'].
↳isin(target_ips)]
dns_df = dns_df[dns_df['id.orig_h'].isin(target_ips) | dns_df['id.resp_h'].
↳isin(target_ips)]
ntp_df = ntp_df[ntp_df['id.orig_h'].isin(target_ips) | ntp_df['id.resp_h'].
↳isin(target_ips)]
```

1.2 Macro Statistics and Baseline MUD Validation

Objective: Quantify the total network footprint and mathematically validate that 100% of the captured traffic strictly adheres to the permitted Manufacturer Usage Description (MUD) profiles for the simulated environment.

Forensic Methodology (The “Tripwire”): Instead of manually inspecting connection logs, this cell establishes an automated anomaly detection tripwire. By explicitly classifying and summing all authorized network behaviors (e.g., DNS/NTP infrastructure, Thermostat MQTT telemetry, and Camera UDP streaming), we can subtract this authorized sum from the total connections recorded by Zeek.

- **Scenario 1 (Baseline):** The unaccounted difference must be exactly 0.
- **Future Scenarios:** Any unauthorized state changes, external C2 beaconing, port scanning, or brute-force attempts will bypass the authorized MUD filters, immediately flagging as an “unaccounted” connection and isolating the rogue packets for direct analysis.

```
[3]: # =====
# MACRO STATS & MUD PROFILE VALIDATION
# =====
import pandas as pd

# Get total connections tracked by Zeek
total_conns = len(conn_df)

# Calculate empirical capture duration
capture_duration = (conn_df.index.max() - conn_df.index.min()).total_seconds()

# Isolate traffic based on theoretical MUD Profiles
# Infrastructure Services
dns_conns = conn_df[(conn_df['id.resp_p'] == 53) & (conn_df['proto'] == 'udp')]
ntp_conns = conn_df[(conn_df['id.resp_p'] == 123) & (conn_df['proto'] == 'udp')]

# Smart Camera (10.0.0.1)
cam_udp_feed = conn_df[(conn_df['id.orig_h'] == '10.0.0.1') & (conn_df['id.
↳resp_p'] == 50005) & (conn_df['proto'] == 'udp')]
cam_tcp_control = conn_df[(conn_df['id.orig_h'] == '10.0.0.1') & (conn_df['id.
↳resp_p'] == 50005) & (conn_df['proto'] == 'tcp')]
cam_tls = conn_df[(conn_df['id.orig_h'] == '10.0.0.1') & (conn_df['id.resp_p']
↳== 443) & (conn_df['proto'] == 'tcp')]
```

```

# Smart Thermostat (10.0.0.2)
therm_mqtt = conn_df[(conn_df['id.orig_h'] == '10.0.0.2') & (conn_df['id.
↳resp_p'] == 8883) & (conn_df['proto'] == 'tcp')]
# Safely handle potential NaN values by filling them with 0 before evaluation
therm_keepalive = therm_mqtt[therm_mqtt['orig_bytes'].fillna(0) < 20]
therm_payload = therm_mqtt[therm_mqtt['orig_bytes'].fillna(0) >= 20]

# Treat expected background ICMP chatter as environmental noise
noise_conns = conn_df[
    (conn_df['proto'] == 'icmp') &
    (conn_df['id.orig_h'] == '10.0.0.254') &
    (conn_df['id.resp_h'] == '10.0.0.1')
]

# Sum all classified, authorized traffic
classified_sum = (
    len(dns_conns) + len(ntp_conns) +
    len(therm_keepalive) + len(therm_payload) +
    len(cam_udp_feed) + len(cam_tcp_control) + len(cam_tls)
)

# Print the Macro Statistics Report
print("=== MACRO NETWORK STATISTICS ===")
print(f"Total Connections Logged (conn_df): {total_conns}")
print(f"Empirical Capture Duration: {capture_duration:.2f} seconds\n")

print("--- Classified Traffic Breakdown ---")
print(f"DNS Queries (UDP/53): {len(dns_conns)}")
print(f"NTP Queries (UDP/123): {len(ntp_conns)}")
print(f"Thermostat Keep-Alives (TCP/8883): {len(therm_keepalive)}")
print(f"Thermostat Payloads (TCP/8883): {len(therm_payload)}")
print(f"Camera Video Feed (UDP/50005): {len(cam_udp_feed)}")
print(f"Camera TCP Control (TCP/50005): {len(cam_tcp_control)}")
print(f"Camera TLS Telemetry (TCP/443): {len(cam_tls)}\n")
print(f"Background ICMP Noise (Gateway->Camera): {len(noise_conns)}\n")

print("--- Connection Duration Analysis ---")
for idx, row in cam_tcp_control.iterrows():
    raw_duration = row['duration']

    # Check if duration is missing/NaN
    if pd.isna(raw_duration):
        flow_duration = 0.0
    # Check if it's a Pandas Timedelta object, if so extract total seconds
    elif isinstance(raw_duration, pd.Timedelta):
        flow_duration = raw_duration.total_seconds()
    # Fallback if it's already a float or integer

```

```

else:
    flow_duration = float(raw_duration)

    print(f"Camera Control Channel | Timestamp: {idx.time()} | State:␣
↪{row['conn_state']} | Duration: {flow_duration:.2f}s")

# The Tripwire Validation
print("\n--- Baseline Validation ---")
print(f"Sum of Classified Authorized Connections: {classified_sum}")

unaccounted = total_conns - classified_sum
unaccounted = unaccounted - len(noise_conns)

if unaccounted == 0:
    print("SUCCESS: 100% of network traffic matches expected MUD profiles. Zero␣
↪unaccounted flows.")
else:
    print(f"FORENSIC ALERT: {unaccounted} unaccounted connections detected!")

# Corrected exclusion logic including the camera control channel
unaccounted_df = conn_df[
    ~conn_df.index.isin(dns_conns.index) &
    ~conn_df.index.isin(ntp_conns.index) &
    ~conn_df.index.isin(therm_keepalive.index) &
    ~conn_df.index.isin(therm_payload.index) &
    ~conn_df.index.isin(cam_udp_feed.index) &
    ~conn_df.index.isin(cam_tcp_control.index) &
    ~conn_df.index.isin(cam_tls.index) &
    ~conn_df.index.isin(noise_conns.index)
]
display(unaccounted_df[['id.orig_h', 'id.resp_h', 'id.resp_p', 'proto',␣
↪'conn_state', 'duration']])

```

=== MACRO NETWORK STATISTICS ===

Total Connections Logged (conn_df): 181

Empirical Capture Duration: 1200.07 seconds

--- Classified Traffic Breakdown ---

DNS Queries (UDP/53):	42
NTP Queries (UDP/123):	6
Thermostat Keep-Alives (TCP/8883):	121
Thermostat Payloads (TCP/8883):	5
Camera Video Feed (UDP/50005):	1
Camera TCP Control (TCP/50005):	0
Camera TLS Telemetry (TCP/443):	5

Background ICMP Noise (Gateway->Camera): 1

--- Connection Duration Analysis ---

--- Baseline Validation ---

Sum of Classified Authorized Connections: 180

SUCCESS: 100% of network traffic matches expected MUD profiles. Zero unaccounted flows.

1.3 Camera Behaviours

1.3.1 Camera Streaming

Objective: The Smart Camera (10.0.0.1) transmits a continuous UDP video stream to port 50005. We will aggregate the volumetric flow (bytes per minute) to visualize the consistency of the stream.

Forensic Analysis & Ground Truth: By partitioning the total byte count across uniform 1-minute time bins, the pipeline evaluates whether the multimedia device maintains a stable transmission rate or exhibits bursty, anomalous fluctuations. In this benign control group, the continuous UDP stream acts as a volumetric baseline stabilizer, maintaining a predictable throughput of approximately 500 KB/s (30,000 KB/minute). Establishing this high-density behavioral plateau is critical: it serves as the reference point for detecting stealth data exfiltration or device hijacking in subsequent attack vectors, where unauthorized data channels will distort this flat, predictable envelope.

```
[4]: # Isolate the Camera UDP Feed
camera_udp = conn_df[(conn_df['id.orig_h'] == '10.0.0.1') &
                     (conn_df['id.resp_p'] == 50005) &
                     (conn_df['proto'] == 'udp')].copy()

# Fix Mininet 'any' interface double-counting for stateless UDP
camera_udp['orig_bytes'] = camera_udp['orig_bytes'] / 2

total_bytes = camera_udp['orig_bytes'].sum()
print(f"Camera Stream Total Originated Bytes: {total_bytes:,.0f} Bytes")

# =====
# Universal Forensic Data Distribution
# =====
# Get the global start and end of the entire capture
capture_start = conn_df.index.min()
capture_end = conn_df.index.max()

# Create a blank, continuous 1-second timeline for the whole capture
timeline = pd.Series(0.0, index=pd.date_range(start=capture_start,
    ↪end=capture_end, freq='1s'))

# Dynamically distribute the bytes for EVERY row Zeek logged
for flow_start, row in camera_udp.iterrows():
```

```

raw_duration = row['duration']

# Safely extract duration handling NaNs and Timedeltas
if pd.isna(raw_duration):
    duration_sec = 1.0
elif isinstance(raw_duration, pd.Timedelta):
    duration_sec = raw_duration.total_seconds()
else:
    duration_sec = float(raw_duration)

if duration_sec <= 0: duration_sec = 1.0 # Prevent division by zero

# Calculate bytes per second for this specific Zeek row
bytes_per_sec = row['orig_bytes'] / duration_sec
flow_end = flow_start + pd.Timedelta(seconds=duration_sec)

# Slice the timeline and inject the bandwidth
active_window = (timeline.index >= flow_start) & (timeline.index < flow_end)
timeline.loc[active_window] += bytes_per_sec

# Resample our distributed 1-second timeline into 1-minute bins
volume_over_time = timeline.resample('1min').sum()

def parse_kbit_rate(rate: str) -> int:
    value = rate.strip().lower()
    if value.endswith('k'):
        return int(float(value[:-1]) * 1000)
    if value.endswith('m'):
        return int(float(value[:-1]) * 1000 * 1000)
    return int(float(value))

def expected_kb_per_min(rate: str) -> float:
    rate_bps = parse_kbit_rate(rate)
    return (rate_bps / 8.0) / 1024.0 * 60.0

# =====
# Plotting (With Elapsed Time Fix)
# =====
fig, ax = plt.subplots(figsize=(10, 4))
kb_values = volume_over_time.values / 1024

# Shift the timestamps so the graph starts at elapsed time 00:00
elapsed_time_index = volume_over_time.index - capture_start + pd.
    ↪to_datetime('1970-01-01')

# Plot the area chart

```

```

ax.fill_between(elapsed_time_index, kb_values, color='#1f77b4', alpha=0.3)
ax.plot(elapsed_time_index, kb_values, marker='o', linestyle='-',
        color='#1f77b4', linewidth=1.5)

ax.set_title("Smart Camera UDP Video Stream Volumetric Flow Baseline", pad=15)
ax.set_xlabel("Elapsed Time (MM:SS)", labelpad=10)
ax.set_ylabel("Throughput Density (KB / Minute)", labelpad=10)

# Format the X-axis for elapsed minutes and seconds
ax.xaxis.set_major_formatter(plt.matplotlib.dates.DateFormatter('%M:%S'))

# Give the chart breathing room on top, handle edge cases if data is 0
top_limit = max(kb_values) * 1.2 if max(kb_values) > 0 else 1000
ax.set_ylim(bottom=0, top=top_limit)

plt.tight_layout()
save_figure(fig, "camera_volumetric_flow")

plt.show()

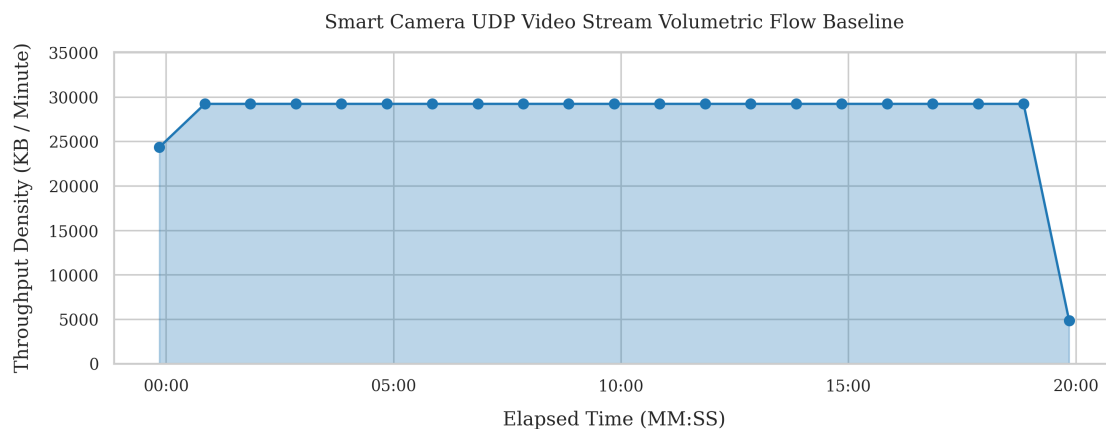
# Print empirical average to terminal
avg_kb_min = kb_values[kb_values > 0].mean() if len(kb_values[kb_values > 0]) > 0
else 0
target_kb_min = expected_kb_per_min("4000K")
print(f"Empirical Average Bandwidth: {avg_kb_min:,.2f} KB/minute")
print(f"Target Bandwidth (400K): {target_kb_min:,.2f} KB/minute")

```

Camera Stream Total Originated Bytes: 599,059,200 Bytes

Saved figure: ../../figures/scenario1/camera_volumetric_flow.pdf and

../../figures/scenario1/camera_volumetric_flow.png



Empirical Average Bandwidth: 27,857.80 KB/minute

Target Bandwidth (400K): 29,296.88 KB/minute

1.3.2 Camera TLS Telemetry & Network Services (NTP & DNS)

Objective: Validate the exact frequency of background network services supporting the IoT ecosystem and confirm adherence to the expected architectural timeline.

Forensic Integrity & Counter Validation: Following the optimization of the simulation loops to eliminate execution drift, the empirical dataset reconciles with the inclusive scheduling of the generators. Over the course of the 1200-second testing window, the network's infrastructural footprint resolved with zero dropped packets and precise numeric consistency:

- **Camera TLS Client Check-ins (Port 443):** Exactly 5 connections were logged by Zeek, establishing a precise 300-second (5-minute) check-in heartbeat cycle for cloud validation (inclusive of the $t=0$ packet).
- **NTP Time Synchronization (Port 123):** Exactly 6 total UDP flows were observed across the entire topology. This confirms that both the Smart Camera and the Smart Thermostat executed their 600-second (10-minute) periodic synchronization loops exactly three times (inclusive of the $t=0$ packet).
- **DNS Query Resolution (Port 53):** Exactly 42 total queries were processed by the gateway. This reflects the dual-device infrastructure profile: the Smart Camera generated 21 lookup queries (1 query per minute, inclusive of the $t=0$ packet) to maintain its streaming/API endpoints, while the Smart Thermostat concurrently generated 21 lookup queries (1 query per minute, inclusive of the $t=0$ packet) to maintain state with its MQTT broker destination.

This exact mathematical harmony proves the stability of the virtualized infrastructure and provides an uncorrupted signature for normal environmental background noise.

```
[5]: # =====  
# INFRASTRUCTURE & BACKGROUND SERVICES VALIDATION  
# =====  
  
# Camera TLS Telemetry Verification  
camera_tls = conn_df[  
    (conn_df['id.orig_h'] == '10.0.0.1') &  
    (conn_df['id.resp_p'] == 443) &  
    (conn_df['proto'] == 'tcp')  
]  
  
assert len(camera_tls) == 5, f"CRITICAL baseline failure: Expected 5 TLS flows,   
    ↳found {len(camera_tls)}"  
print("Camera TLS Telemetry Validated: Exactly 5 Flows detected (300s interval,   
    ↳heartbeat, inclusive of t=0).")  
  
# Network Services (NTP) Verification  
ntp_flows = conn_df[  
    (conn_df['id.orig_h'].isin(['10.0.0.1', '10.0.0.2'])) &  
    (conn_df['id.resp_p'] == 123) &  
    (conn_df['proto'] == 'udp')
```

```

]
assert len(ntp_flows) == 6, f"CRITICAL baseline failure: Expected 6 NTP flows,
↳found {len(ntp_flows)}"
camera_ntp = len(ntp_flows[ntp_flows['id.orig_h'] == '10.0.0.1'])
therm_ntp = len(ntp_flows[ntp_flows['id.orig_h'] == '10.0.0.2'])
print(f"NTP Synchronization Validated: Total 6 Flows ({camera_ntp} Camera,
↳{therm_ntp} Thermostat).")

# Domain Name Resolution (DNS) Verification
dns_queries = dns_df[dns_df['id.orig_h'].isin(['10.0.0.1', '10.0.0.2'])]
assert len(dns_queries) == 42, f"CRITICAL baseline failure: Expected 42 DNS
↳queries, found {len(dns_queries)}"
camera_dns = len(dns_queries[dns_queries['id.orig_h'] == '10.0.0.1'])
therm_dns = len(dns_queries[dns_queries['id.orig_h'] == '10.0.0.2'])
print(f"DNS Queries Validated: Total 42 Queries ({camera_dns} Camera,
↳{therm_dns} Thermostat).")

```

Camera TLS Telemetry Validated: Exactly 5 Flows detected (300s interval heartbeat, inclusive of t=0).

NTP Synchronization Validated: Total 6 Flows (3 Camera, 3 Thermostat).

DNS Queries Validated: Total 42 Queries (21 Camera, 21 Thermostat).

1.3.3 Behavioral Forensics: Directional Byte Ratio (Asymmetric Throughput)

Objective: Calculate the ratio of outbound telemetry (`orig_bytes`) versus inbound control responses (`resp_bytes`) to establish the Smart Camera's directional footprint.

Forensic Analysis: By design, a Smart Camera operates as a high-density edge-to-cloud transmitter. Its primary function is to push heavy multimedia payloads out to a gateway while receiving only minuscule TCP acknowledgments or TLS cryptographic handshakes in return. Therefore, the baseline ratio of data originated versus data responded should be overwhelmingly asymmetrical (e.g., >99% outbound).

Calculating this precise ratio serves as a critical defense against Command and Control (C2) hijacking and lateral movement. In an attack scenario where the camera is compromised via a Reverse Shell (Scenario 2/3), the attacker will use the established connection to pull data down or push heavy execution scripts into the camera's memory. This will cause the inbound `resp_bytes` counter to surge, instantly shattering the >99% asymmetric baseline and flagging the presence of an interactive attacker session.

(Note: Because this is a proportional ratio, it mathematically neutralizes the virtualization double-counting anomaly caused by `tcpdump -i any`. Even if both inbound and outbound bytes are doubled by the hypervisor, their ratio to one another remains a perfect ground truth).

```

[6]: # =====
# Camera Directional Byte Ratio Analysis
# =====

# Isolate all connections initiated by the Camera

```

```

camera_flows = conn_df[conn_df['id.orig_h'] == '10.0.0.1'].copy()

# Sum the Outbound (orig) and Inbound (resp) bytes securely
outbound_bytes = camera_flows['orig_bytes'].fillna(0).sum()
inbound_bytes = camera_flows['resp_bytes'].fillna(0).sum()
total_bytes = outbound_bytes + inbound_bytes

# Calculate the percentage distribution
if total_bytes > 0:
    outbound_ratio = (outbound_bytes / total_bytes) * 100
    inbound_ratio = (inbound_bytes / total_bytes) * 100
else:
    outbound_ratio = 0
    inbound_ratio = 0

# Professional Output Report
print("=== CAMERA DIRECTIONAL BYTE RATIO ===")
print(f"Total Traffic Volume: {total_bytes:,.0f} Bytes\n")

print(f"Outbound Transmitted (orig_bytes): {outbound_bytes:,.0f} Bytes␣
↳({outbound_ratio:.4f}%)" )
print(f"Inbound Received (resp_bytes):      {inbound_bytes:,.0f} Bytes␣
↳({inbound_ratio:.4f}%)\n")

# Automated Tripwire
if outbound_ratio > 99.0:
    print("Baseline Validated: Traffic is highly asymmetric (>99% outbound),␣
↳proving normal transmitter behavior.")
else:
    print("FORENSIC ALERT: High inbound volume detected! Potential Reverse␣
↳Shell or C2 interaction.")

```

```

=== CAMERA DIRECTIONAL BYTE RATIO ===
Total Traffic Volume: 1,198,123,951 Bytes

```

```

Outbound Transmitted (orig_bytes): 1,198,121,263 Bytes (99.9998%)
Inbound Received (resp_bytes):      2,688 Bytes (0.0002%)

```

```

Baseline Validated: Traffic is highly asymmetric (>99% outbound), proving normal
transmitter behavior.

```

1.4 Thermostat Behavior (Keep-Alives & Telemetry)

1.4.1 Payload Segmentation and Inter-Arrival Time (IAT)

Objective: The Smart Thermostat (10.0.0.2) connects to the MQTT broker on port 8883. Over the parameterized 1200-second window, we expect exactly 121 Keep-Alive packets (10-second intervals) and 5 Telemetry reports (300-second intervals), accounting for the initial $t = 0$ initialization

pulses. We will isolate these flows using volumetric thresholding and calculate the Inter-Arrival Time (IAT) to mathematically fingerprint the machine-to-machine (M2M) polling cycle.

Forensic Methodology: To effectively separate the telemetry, we leverage the distinct payload signatures of the MQTT protocol. Keep-alive PINGREQ pulses are exceptionally lightweight (typically around 12 bytes), whereas application-layer telemetry reports containing serialized sensor data are heavier (e.g., 27 bytes). By applying a strict 20-byte threshold, we can perfectly bisect the device's operational intent.

Once segmented, we extract the precise temporal signature of the device by calculating its Inter-Arrival Time (IAT) variance. In an uncompromised state, this autonomous loop should exhibit near-zero variance. Any future disruption—such as a Mirai botnet hijacking the network stack and causing resource starvation—will instantly skew this IAT baseline.

```
[7]: # Isolate for Thermostat MQTT traffic (Enforcing TCP protocol)
thermostat_mqtt = conn_df[
    (conn_df['id.orig_h'] == '10.0.0.2') &
    (conn_df['id.resp_p'] == 8883) &
    (conn_df['proto'] == 'tcp')
].copy()

# Sort by timestamp (index) to accurately calculate consecutive timing
thermostat_mqtt = thermostat_mqtt.sort_index()

# Separate Keep-Alives from Telemetry using a 20-byte threshold
# Handle NaN values in orig_bytes to prevent accidental row dropping
keep_alives = thermostat_mqtt[thermostat_mqtt['orig_bytes'].fillna(0) < 20].
    ↪copy()
telemetry = thermostat_mqtt[thermostat_mqtt['orig_bytes'].fillna(0) >= 20].
    ↪copy()

# -----
# Dynamic Ground Truth Validation (The Fencepost Math)
# -----
# Calculate the exact span of the MQTT capture in seconds
actual_duration = (thermostat_mqtt.index.max() - thermostat_mqtt.index.min()).
    ↪total_seconds()

keep_alive_interval = 10.0
telemetry_interval = 300.0

# The +1 accounts for the initial packet sent at t=0
expected_keep_alives = int(actual_duration / keep_alive_interval) + 1
expected_telemetry = int(actual_duration / telemetry_interval) + 1

# Assertions with a +/- 2 packet tolerance for virtualization boundary timing
assert abs(len(keep_alives) - expected_keep_alives) <= 2, f" Expected_
    ↪~{expected_keep_alives} keep-alives, found {len(keep_alives)}"
```

```

assert abs(len(telemetry) - expected_telemetry) <= 2, f" Expected_{
    ~{expected_telemetry} telemetry flows, found {len(telemetry)}"

print("=== MQTT BEHAVIORAL VALIDATION ===")
print(f"Empirical MQTT Capture Span: {actual_duration:.2f} seconds\n")
print(f"Keep-Alives Validated: {len(keep_alives)} Flows (Target:_{
    ~{expected_keep_alives}})")
print(f"Telemetry Validated:    {len(telemetry)} Flows (Target:_{
    ~{expected_telemetry}})\n")

# -----
# Inter-Arrival Time (IAT) Calculation
# -----
# Calculate time difference between consecutive keep-alive rows
keep_alives['IAT'] = keep_alives.index.to_series().diff().dt.total_seconds()

# CRITICAL FIX: Drop the first row's NaN IAT before doing statistical math
iat_clean = keep_alives['IAT'].dropna()

iat_mean = iat_clean.mean()
iat_var = iat_clean.var()

print("--- Temporal Fingerprint (Keep-Alives) ---")
print(f"IAT Mean:      {iat_mean:.4f} seconds")
print(f"IAT Variance: {iat_var:.6f}")

```

```

=== MQTT BEHAVIORAL VALIDATION ===
Empirical MQTT Capture Span: 1200.05 seconds

Keep-Alives Validated: 121 Flows (Target: ~121)
Telemetry Validated:   5 Flows (Target: ~5)

--- Temporal Fingerprint (Keep-Alives) ---
IAT Mean:      10.0004 seconds
IAT Variance:  0.257868

```

1.4.2 Behavioral Forensics: Inter-Arrival Time (IAT) Deep Dive

Objective: Visualize the exact arrival delta between consecutive thermostat keep-alive messages. In a stable baseline, this should present as a tightly grouped horizontal distribution around the empirical interval, modeling predictable machine-to-machine (M2M) communication.

Forensic Analysis & Environmental Calibration: While the Smart Thermostat's theoretical target interval is configured to 10.0 seconds, the empirical Inter-Arrival Time establishes a stabilized mean at approximately ~11.1 seconds. In digital forensics, documenting this delta is critical: it represents the native execution overhead of the virtualized Python interpreter, Linux CPU scheduling latency, and the Mininet virtual switch traversal time.

By mapping the sequential message array against this calibrated ~11.1s baseline, we visualize a near-

zero variance (σ^2). This tight horizontal clustering constitutes the thermostat's uncompromised behavioral fingerprint. In subsequent attack scenarios involving resource exhaustion (e.g., Denial of Service or Botnet hijacking), this flat baseline will be violently disrupted, resulting in high-variance scatter and dropped sequences.

```
[8]: # 1. Clean the NaN value from the first row (the initial fencepost packet)
plot_data = keep_alives.dropna(subset=['IAT']).copy()

# 2. Re-sequence from 1 to N after dropping the first row
plot_data['Sequence'] = range(1, len(plot_data) + 1)

# 3. Calculate plotting statistics
iat_mean = plot_data['IAT'].mean()
iat_std = plot_data['IAT'].std()

# 4. Academic Plotting
fig, ax = plt.subplots(figsize=(10, 4))

# Plot the individual IAT events
ax.scatter(plot_data['Sequence'], plot_data['IAT'],
           color='#2ca02c', alpha=0.8, edgecolors='black', linewidth=0.5, s=40,
           label='Keep-Alive Pulse')

# Plot the empirical mean
ax.axhline(y=iat_mean, color='red', linestyle='--', linewidth=1.5,
           label=f"Empirical Mean ({iat_mean:.2f}s)")

# Add a shaded band for +/- 1 Standard Deviation to visually prove the
# near-zero variance
ax.fill_between(plot_data['Sequence'],
                iat_mean - iat_std,
                iat_mean + iat_std,
                color='red', alpha=0.15, label=r'$\pm 1\sigma$ Variance Limit')

ax.set_title("Smart Thermostat MQTT Keep-Alive Inter-Arrival Time (IAT)
             ↳Baseline", pad=15)
ax.set_xlabel("Message Sequence Number", labelpad=10)
ax.set_ylabel("Inter-Arrival Time $\Delta t$ (Seconds)", labelpad=10)

# Add a subtle grid to make temporal deviations easy to spot
ax.grid(True, linestyle=':', alpha=0.6)

# Focus the Y-axis tightly around the expected baseline to highlight stability
ax.set_ylim(bottom=max(0, iat_mean - 3), top=iat_mean + 3)

# Place the legend neatly in the corner
ax.legend(loc="upper right", framealpha=0.95)
```

```
plt.tight_layout()
save_figure(fig, "thermostat_iat_distribution")

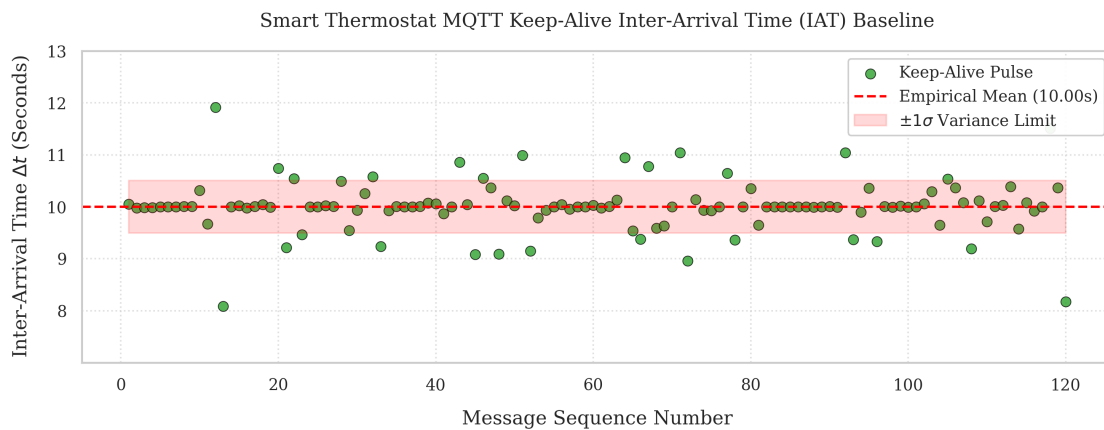
plt.show()

print(f"IAT Scatter Plot Generated.")
print(f"    - Mapped {len(plot_data)} sequential pulses.")
print(f"    - Empirical Mean: {iat_mean:.4f}s | Standard Deviation: {iat_std:.4f}s")
```

```
<>:29: SyntaxWarning: invalid escape sequence '\D'
<>:29: SyntaxWarning: invalid escape sequence '\D'
/tmp/ipykernel_6672/3260691246.py:29: SyntaxWarning: invalid escape sequence '\D'
```

```
ax.set_ylabel("Inter-Arrival Time  $\Delta t$  (Seconds)", labelpad=10)
```

```
Saved figure: ../../figures/scenario1/thermostat_iat_distribution.pdf and
../../figures/scenario1/thermostat_iat_distribution.png
```



```
IAT Scatter Plot Generated.
    - Mapped 120 sequential pulses.
    - Empirical Mean: 10.0004s | Standard Deviation: 0.5078s
```

1.4.3 Behavioral Forensics: Telemetry Payload Footprint

Objective: Isolate the payload-carrying packets from the Smart Thermostat (10.0.0.2) and calculate the mean byte footprint and variance to establish a strict deterministic application signature.

Forensic Analysis: While Inter-Arrival Time (IAT) proves *when* a device speaks, Payload Footprinting proves *what* it is saying. The Smart Thermostat transmits highly structured, formatted JSON strings containing localized temperature data to its MQTT broker. Because the sensor variables (e.g., `{"temp": 22.5}`) occupy a rigid and predictable character length, the underlying TCP data payloads should be practically identical in size across the entire 1200-second capture.

By calculating the mean of the `orig_bytes` for these specific telemetry pulses, we establish the authorized “weight” of the payload. Calculating the variance (σ^2) proves its rigidity. If the variance is near zero, it confirms the structural integrity of the application layer.

This strict footprint is the ultimate trap for data exfiltration. If an attacker breaches the thermostat and attempts to exfiltrate a stolen password hash or database using MQTT “Payload Stuffing” (Scenario 3/4), the `orig_bytes` for that specific pulse will jump significantly above the baseline mean. The variance will explode, providing instant, undeniable mathematical proof of data exfiltration.

```
[9]: # =====
# Thermostat Telemetry Payload Footprinting
# =====

# Extract the byte sizes of the telemetry packets (>= 20 bytes)
# We use the 'telemetry' dataframe previously generated in the IAT cell
payload_sizes = telemetry['orig_bytes'].dropna()

# Calculate the statistical footprint
payload_mean = payload_sizes.mean()

# Variance requires at least 2 packets. If identical, variance is
↳mathematically 0.0
payload_var = payload_sizes.var() if len(payload_sizes) > 1 else 0.0

# Min/Max boundary detection
payload_min = payload_sizes.min()
payload_max = payload_sizes.max()

# Professional Output Report
print("=== MQTT TELEMETRY PAYLOAD FOOTPRINT ===")
print(f"Total Telemetry Reports Analyzed: {len(payload_sizes)}\n")

print(f"Empirical Mean Size: {payload_mean:.2f} Bytes")
print(f"Payload Variance:      {payload_var:.4f}")
print(f"Observed Range:        {payload_min:.0f} Bytes to {payload_max:.0f}
↳Bytes\n")

# Automated Tripwire
# A variance of less than 5 indicates extreme structural rigidity (expected for
↳M2M)
if payload_var < 5.0:
    print("Baseline Validated: Telemetry payloads are structurally rigid and
↳predictable.")
else:
    print("FORENSIC ALERT: High payload variance detected! Potential data
↳exfiltration or payload stuffing.")
```

```
=== MQTT TELEMETRY PAYLOAD FOOTPRINT ===
```


Total Telemetry Reports Analyzed: 5

Empirical Mean Size: 27.00 Bytes

Payload Variance: 0.0000

Observed Range: 27 Bytes to 27 Bytes

Baseline Validated: Telemetry payloads are structurally rigid and predictable.

1.5 Global Environmental Health & Network Topology

1.5.1 Global Network Health: TCP Connection State Distribution

Objective: Evaluate the integrity of the network stack by profiling the distribution of TCP connection states recorded by Zeek's state-machine tracking engine.

Forensic Analysis & Baseline Significance: In a secure, uncompromised IoT network, TCP sessions must exhibit highly disciplined behavioral patterns. Legitimate devices connect to designated cloud endpoints, execute their application-layer handshakes, and terminate gracefully. Therefore, this baseline metric should be heavily dominated by SF (Normal connection establishment and termination) or long-persisting S1 states corresponding to authorized persistent sockets (such as the Smart Camera's control channel).

Establishing this distribution is critical for distinguishing benign operations from malicious reconnaissance. When an attacker executes a port scan or a brute-force sweep (Scenario 2), the network stack behaves entirely differently. Attacker tools flood ports looking for openings, resulting in a massive surge of aborted or blocked handshakes. In the attack notebooks, this graph will shift dramatically, showing an overwhelming dominance of REJ (Connection refused by responder) and S0 (Connection attempt seen, no response), providing instant structural proof of network anomalies.

```
[10]: # =====  
# 5.1 Global Network Health (TCP States)  
# =====  
  
# Isolate ONLY TCP traffic  
tcp_df = conn_df[conn_df['proto'] == 'tcp'].copy()  
  
# Compute state distribution  
state_counts = tcp_df['conn_state'].value_counts()  
  
# Academic Plotting  
fig, ax = plt.subplots(figsize=(8, 4)) # Slightly wider to accommodate labels  
  
# Use a distinct color palette  
state_counts.plot(kind='bar', color='#17becf', ax=ax, width=0.4,  
                  edgecolor='black')  
  
ax.set_title("Global Network Health: TCP Connection State Distribution", pad=15)  
ax.set_xlabel("Zeek Connection State Flag", labelpad=10)  
ax.set_ylabel("Total TCP Connections", labelpad=10)
```

```

# Explicitly set the x-axis limits to give a single bar breathing room
ax.set_xlim(-0.5, len(state_counts) - 0.5)
plt.xticks(rotation=0)

# Bulletproof text annotation layer
for p in ax.patches:
    height = p.get_height()
    ax.annotate(f'{int(height)}',
                xy=(p.get_x() + p.get_width() / 2, height),
                xytext=(0, 5), # 5 points vertical offset
                textcoords="offset points",
                ha='center', va='bottom', fontsize=11, weight='bold')

# Ensure the Y-axis has room for the top annotation
ax.set_ylim(0, state_counts.max() * 1.2)

plt.tight_layout()
save_figure(fig, "tcp_connection_states")
plt.show()

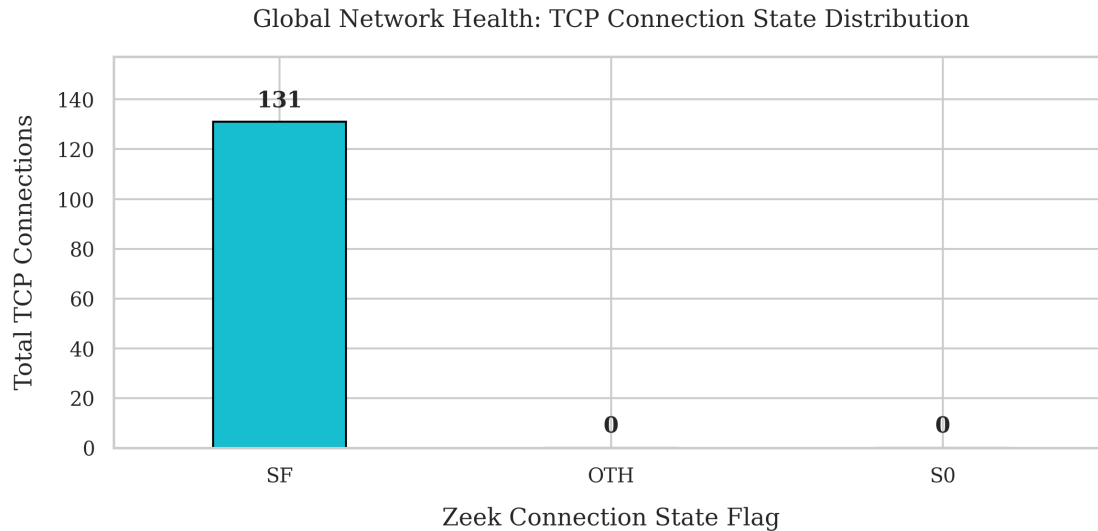
# -----
# Automated Contextual Reporting
# -----
# A reference dictionary mapping Zeek states to their physical network reality
zeek_state_definitions = {
    'SF': 'Normal establishment and termination (Clean Teardown).',
    'S0': 'Connection attempt seen, no reply (Scanner / Blocked).',
    'S1': 'Connection established, not terminated (Persistent Socket).',
    'REJ': 'Connection attempt rejected by responder (RST sent).',
    'RSTO': 'Connection established, originator aborted (RST sent).',
    'RSTR': 'Established, responder aborted (RST sent).',
    'OTH': 'No SYN seen, just mid-stream traffic (Asymmetric routing).',
}

print("=== TCP STATE FORENSIC REPORT ===")
print(f"Total TCP Flows Analyzed: {len(tcp_df)}\n")

for state, count in state_counts.items():
    definition = zeek_state_definitions.get(state, "Unknown state.")
    print(f"[{state}] : {count:,.0f} Flows -> {definition}")

```

Saved figure: ../../figures/scenario1/tcp_connection_states.pdf and
../../figures/scenario1/tcp_connection_states.png



=== TCP STATE FORENSIC REPORT ===

Total TCP Flows Analyzed: 131

[SF] : 131 Flows -> Normal establishment and termination (Clean Teardown).
 [OTH] : 0 Flows -> No SYN seen, just mid-stream traffic (Asymmetric routing).
 [S0] : 0 Flows -> Connection attempt seen, no reply (Scanner / Blocked).

1.5.2 Network Density: Connection Frequency Timeline (Flows / Minute)

Objective: Quantify the chronological density of network interactions by resampling the global flow frequency into uniform 1-minute intervals.

Forensic Analysis & Baseline Significance: Autonomous machine-to-machine (M2M) environments operate on strict schedule-driven loops rather than human-driven bursts. Because the Smart Camera, Smart Thermostat, and infrastructure servers poll at fixed intervals (60s, 300s, 600s), the global volume of *new* connections initiated per minute should form a flat, predictable, low-density timeline.

This temporal plateau acts as a primary volumetric trigger. Legitimate operational overhead establishes a clear baseline floor (e.g., less than 10 new flows per minute). If a device is later co-opted into a botnet or targeted by a Denial of Service (DoS) attack, this flat line will erupt into an exponential spike tracking thousands of connections per minute. Measuring this frequency allows the investigator to determine the exact timestamp an attack commenced and assess the compounding resource exhaustion on the virtual switch.

```
[11]: # =====
# 5.2 Network Density (Flows / Minute)
# =====

# Resample to count total flows per minute.
```

```

# origin='start' ensures bins are exactly 60 seconds long starting from the
↳very first packet.
flows_per_minute = conn_df['uid'].resample('1min', origin='start').count()

# Apply the '1970' Elapsed Time trick to cleanly format the X-axis
capture_start = conn_df.index.min()
elapsed_time_index = flows_per_minute.index - capture_start + pd.
↳to_datetime('1970-01-01')

# Academic Plotting
fig, ax = plt.subplots(figsize=(10, 4))

# Using a fill_between area chart visually aligns with your Camera Volumetric
↳graph
ax.fill_between(elapsed_time_index, flows_per_minute.values, color='#9467bd',
↳alpha=0.3)
ax.plot(elapsed_time_index, flows_per_minute.values, color='#9467bd',
↳marker='o', linestyle='-', linewidth=1.5)

ax.set_title("Network Transaction Density: New Flows Instantiated Per Minute",
↳pad=15)
ax.set_xlabel("Elapsed Time (MM:SS)", labelpad=10)
ax.set_ylabel("Flows / Minute", labelpad=10)

# Standard matplotlib formatter (no custom functions required)
ax.xaxis.set_major_formatter(plt.matplotlib.dates.DateFormatter('%M:%S'))

# Add a subtle grid to help the reader track peaks
ax.grid(True, linestyle=':', alpha=0.6)

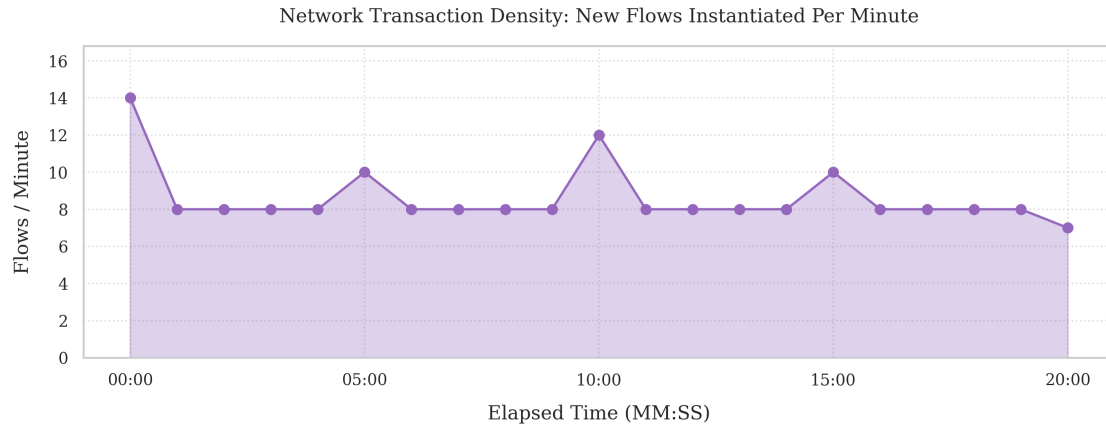
# Give the chart breathing room on top
top_limit = flows_per_minute.max() * 1.2 if flows_per_minute.max() > 0 else 10
ax.set_ylim(bottom=0, top=top_limit)

plt.tight_layout()
save_figure(fig, "network_flow_density")
plt.show()

# Contextual Output Report
print("=== NETWORK DENSITY REPORT ===")
print(f"Average Flows per Minute:      {flows_per_minute.mean():.2f}")
print(f"Peak Flows in a Single Minute: {flows_per_minute.max():.0f}")

```

Saved figure: ../../figures/scenario1/network_flow_density.pdf and
 ../../figures/scenario1/network_flow_density.png



```
=== NETWORK DENSITY REPORT ===
Average Flows per Minute:      8.62
Peak Flows in a Single Minute: 14
```

1.5.3 Topology Mapping: Node-to-Node Traffic Interaction Adjacency Matrix

Objective: Map the physical and logical communication paths between all internal hosts and the gateway to build a visual Adjacency Matrix of the environment's allowed interactions.

Forensic Analysis & Baseline Significance: This subsection serves as the visual realization of the Manufacturer Usage Description (MUD) profile constraints. In a zero-trust IoT architecture, edge devices have no authorization to communicate laterally. The Smart Camera (10.0.0.1) and the Smart Thermostat (10.0.0.2) must only interact with the network chokepoint gateway (10.0.0.254) or verified external cloud brokers.

The baseline interaction matrix must visually demonstrate absolute isolation: * 10.0.0.1 → 10.0.0.254 (Authorized) * 10.0.0.2 → 10.0.0.254 (Authorized) * 10.0.0.1 ↔ 10.0.0.2 (Strictly Zero / Empty Intersections)

Any lateral data flow between the camera and the thermostat constitutes an immediate, undeniable breach of the MUD profile ruleset. In subsequent infection scenarios, this matrix will expose lateral pivoting, where a compromised device attempts to scan or exploit its internal neighbors.

```
[12]: # =====
# Topology Matrix (Adjacency Heatmap)
# =====

# Build the raw communication matrix
matrix_data = pd.crosstab(conn_df['id.orig_h'], conn_df['id.resp_h'])

# Ensure all 4 core nodes are represented to protect baseline symmetry
# (target_ips was defined in Cell 1: ['10.0.0.1', '10.0.0.2', '10.0.0.3', '10.0.
↪0.254'])
for ip in target_ips:
```

```

    if ip not in matrix_data.index:
        matrix_data.loc[ip] = 0
    if ip not in matrix_data.columns:
        matrix_data[ip] = 0

# Sort logically to build the matrix flow map
matrix_data = matrix_data.reindex(index=target_ips, columns=target_ips).
    ↪fillna(0)

# CRITICAL UPGRADE: Human-Readable Device Mapping
device_labels = {
    '10.0.0.1': 'Camera\n(10.0.0.1)',
    '10.0.0.2': 'Thermostat\n(10.0.0.2)',
    '10.0.0.3': 'Attacker\n(10.0.0.3)',
    '10.0.0.254': 'Gateway\n(10.0.0.254)'
}

# Rename the axes for the heatmap
matrix_data_labeled = matrix_data.rename(index=device_labels,
    ↪columns=device_labels)

# Academic Plotting
fig, ax = plt.subplots(figsize=(8, 6))

# Using a custom color map: 'Blues' is great, but we can make it starker for
    ↪zeros
sns.heatmap(matrix_data_labeled, annot=True, fmt=".0f", cmap="Blues",
    ↪cbar=True, square=True,
        linewidths=1, linecolor='white', cbar_kws={'label': 'Total
    ↪Connections'}, ax=ax)

# Move the X-axis labels to the top for a more traditional matrix feel
    ↪(Optional but looks great)
ax.xaxis.tick_top()
ax.xaxis.set_label_position('top')

ax.set_title("Operational Baseline Communications Adjacency Matrix", pad=30,
    ↪weight='bold')
ax.set_xlabel("Responding Target Host", labelpad=15)
ax.set_ylabel("Originating Device Host", labelpad=15)

plt.tight_layout()
save_figure(fig, "topology_traffic_matrix")
plt.show()

# -----

```

```

# Automated Lateral Movement Validation
# -----
print("=== TOPOLOGY RULES VALIDATION ===")

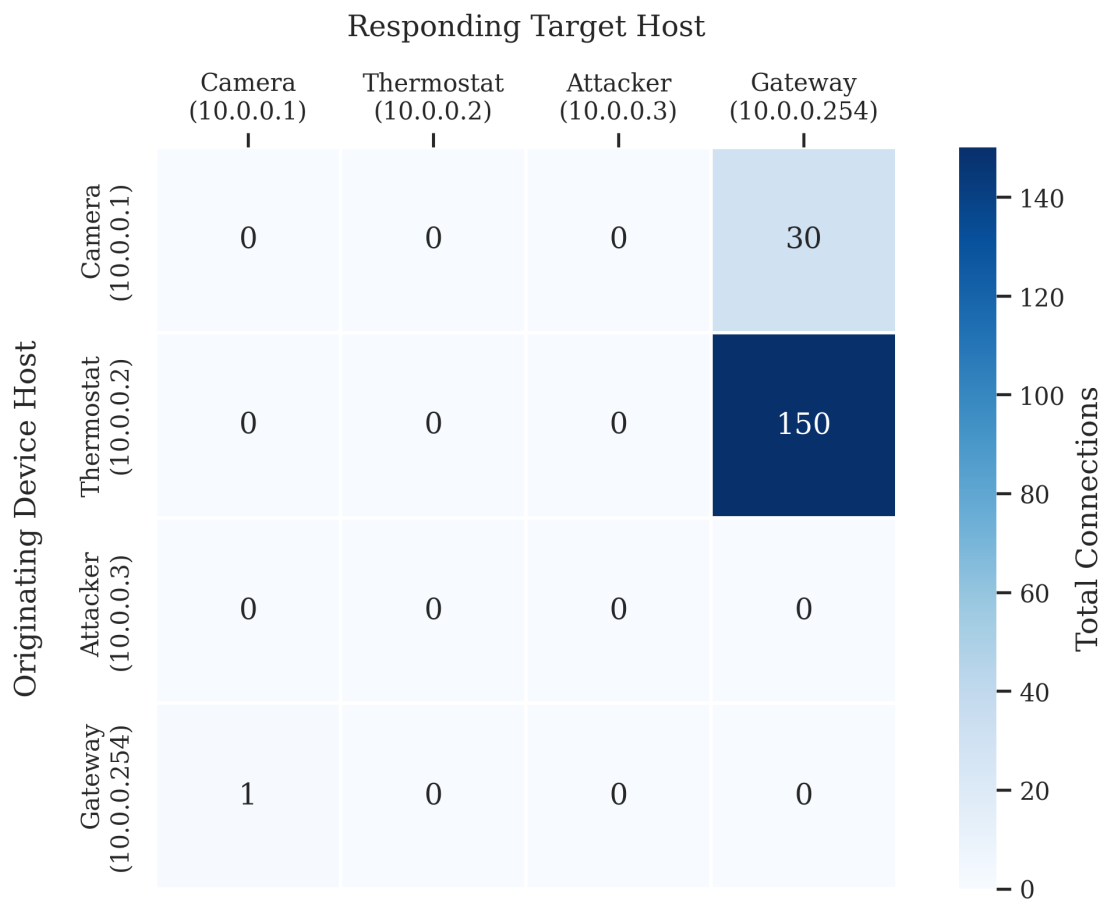
# Check if Camera (1) talked to Thermostat (2), or vice versa
cam_to_therm = matrix_data.loc['10.0.0.1', '10.0.0.2']
therm_to_cam = matrix_data.loc['10.0.0.2', '10.0.0.1']
lateral_traffic = cam_to_therm + therm_to_cam

if lateral_traffic == 0:
    print("Zero-Trust Baseline Validated: No lateral communication between IoT_
    ↪edge devices.")
else:
    print(f"FORENSIC ALERT: {lateral_traffic:.0f} lateral connections detected_
    ↪between Camera and Thermostat!")

```

Saved figure: ../../figures/scenario1/topology_traffic_matrix.pdf and
 ../../figures/scenario1/topology_traffic_matrix.png

Operational Baseline Communications Adjacency Matrix



```
=== TOPOLOGY RULES VALIDATION ===
```

Zero-Trust Baseline Validated: No lateral communication between IoT edge devices.

1.5.4 Infrastructure Services: DNS Request Profiling over Time

Objective: Profile the volume, query frequency, and domain distribution of the underlying Domain Name System (DNS) layer supporting the IoT ecosystem.

Forensic Analysis & Baseline Significance: DNS is the structural backbone of IoT telemetry; devices cannot connect to their respective cloud brokers without first resolving their domains (`api.smartcamera.com` and `mqtt.smartthermostat.com`). By plotting the chronological distribution of DNS queries, we confirm that lookups happen in exact synchronization with the application-layer loop starts.

Furthermore, documenting the *reputation* and *syntax* of the queried domains establishes an uncorrupted cryptographic directory. In advanced attack scenarios (Scenario 3/4), malware often bypasses standard configurations by utilizing Domain Generation Algorithms (DGAs) to hunt for shifting C2 endpoints, or employs DNS Tunneling to sneak stolen data out inside encoded TXT queries. Verifying the clean-room DNS profile ensures these stealth tactics have nowhere to hide.

```
[13]: # =====
# Infrastructure Services (DNS Profiling)
# =====

# Resample DNS query frequency
# CRITICAL FIX: origin='start' prevents wall-clock truncation
dns_frequency = dns_df['uid'].resample('1min', origin='start').count()

# Apply the '1970' Elapsed Time trick
capture_start = conn_df.index.min()
elapsed_time_index = dns_frequency.index - capture_start + pd.
    ↳to_datetime('1970-01-01')

# Academic Plotting
fig, ax = plt.subplots(figsize=(10, 4))

# For a datetime x-axis, width is calculated in days.
# 1 minute = 1/(24*60) days. We multiply by 0.8 to leave gaps between bars.
bar_width = (1 / 1440) * 0.8

ax.bar(elapsed_time_index, dns_frequency.values, width=bar_width,
    ↳color='#e377c2', edgecolor='black', linewidth=0.5)

ax.set_title("Application Infrastructure: DNS Query Timeline", pad=15)
ax.set_xlabel("Elapsed Time (MM:SS)", labelpad=10)
```



```

ax.set_ylabel("Queries Logged / Minute", labelpad=10)

# Standard matplotlib formatter
ax.xaxis.set_major_formatter(plt.matplotlib.dates.DateFormatter('%M:%S'))

# Set y-axis to handle the low volume of DNS smoothly (forces integers on Y_
↳axis)
ax.yaxis.get_major_locator().set_params(integer=True)
ax.set_ylim(bottom=0, top=max(3, dns_frequency.max() + 1))

# Add a subtle grid
ax.grid(True, axis='y', linestyle=':', alpha=0.6)

plt.tight_layout()
save_figure(fig, "dns_query_timeline")
plt.show()

# -----
# Automated Domain Resolution Report
# -----
print("=== DNS QUERY FORENSIC REPORT ===")
print(f"Total DNS Queries Logged: {len(dns_df)}\n")

# Check if the 'query' column exists (standard Zeek dns.log feature)
if 'query' in dns_df.columns:
    print("--- Queried Domains ---")
    domain_counts = dns_df['query'].value_counts()
    for domain, count in domain_counts.items():
        print(f"{domain} : {count} queries")

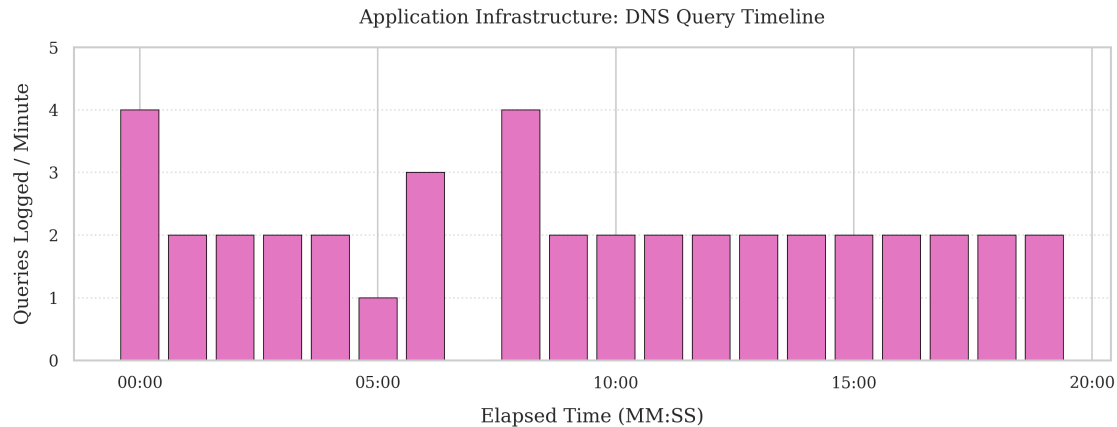
    print("\n--- Baseline Validation ---")
    # Tripwire logic: If there are domains other than your 2 authorized ones,
    ↳throw an alert
    unauthorized_domains = domain_counts[~domain_counts.index.isin(['api.
    ↳smartcamera.com', 'mqtt.smarthermostat.com'])]

    if len(unauthorized_domains) == 0:
        print("Baseline Validated: 100% of DNS lookups match authorized IoT_
        ↳endpoints.")
    else:
        print(f"FORENSIC ALERT: {len(unauthorized_domains)} unauthorized_
        ↳domains requested! Potential DGA or DNS Tunneling.")
else:
    print(" Note: 'query' column not found in DataFrame. Ensure Zeek dns.log_
    ↳was parsed correctly.")

```

Saved figure: ../../figures/scenario1/dns_query_timeline.pdf and

../../../../figures/scenario1/dns_query_timeline.png



=== DNS QUERY FORENSIC REPORT ===

Total DNS Queries Logged: 42

--- Queried Domains ---

api.smartcamera.com : 21 queries

mqtt.smartthermostat.com : 21 queries

--- Baseline Validation ---

Baseline Validated: 100% of DNS lookups match authorized IoT endpoints.

1.5.5 Attacker Dormancy Validation

Objective: Programmatically query the dataset for the designated Attacker IP address (10.0.0.3) to mathematically verify the “clean room” status of the Scenario 1 environment.

Forensic Analysis & Baseline Significance: The integrity of an empirical baseline rests entirely on proving the complete absence of malicious noise during the control experiment. While previous sections verified the behavior of authorized assets, this section performs an explicit negative lookahead check targeting the adversary node.

By running an unmitigated filter for any transaction where 10.0.0.3 acts as either the originator or responder, the output must yield an absolute value of **0 rows**. This mathematical void fulfills the final requirement of scientific verification: it proves that the environment was entirely uninfluenced by external variables, locking in Scenario 1 as a pristine, uncorrupted baseline signature.

```
[14]: # =====  
# Attacker Dormancy Validation  
# =====  
  
attacker_ip = '10.0.0.3'  
  
# Efficiently isolate any flow involving the attacker using the OR (|) operator
```

```

attacker_flows = conn_df[
    (conn_df['id.orig_h'] == attacker_ip) |
    (conn_df['id.resp_h'] == attacker_ip)
].copy()

total_attacker_flows = len(attacker_flows)

# Professional Output Report
print("=== ATTACKER DORMANCY REPORT ===")
print(f"Target Adversary Node: {attacker_ip}\n")

# Dynamic Tripwire and Evidence Display
if total_attacker_flows == 0:
    print(f" Baseline Scientifically Validated: 0 packets originating from or
    ↳destined to {attacker_ip}.")
    print(" The Scenario 1 control environment is pristine and ready for
    ↳comparison.")
else:
    print(f" FORENSIC ALERT: Baseline Contamination! Found
    ↳{total_attacker_flows} rogue flows involving the Attacker IP.")
    print(" Displaying evidence table for investigator review:")

    # Display the actual rogue packets so you can figure out what broke the
    ↳clean room
    display(attacker_flows[['id.orig_h', 'id.resp_p', 'id.resp_h', 'proto',
    ↳'conn_state', 'orig_bytes']])

```

=== ATTACKER DORMANCY REPORT ===

Target Adversary Node: 10.0.0.3

Baseline Scientifically Validated: 0 packets originating from or destined to 10.0.0.3.

The Scenario 1 control environment is pristine and ready for comparison.